DoMS

QUIET EXCELLENCE DEMONSTRATED

MS3520 : Operations Management I

Operations Scheduling

Introduction-What is Operations Scheduling ?

- Implement the production orders generated in MRP under given objectives ;
- Allocate production resources (machine, workers et al.) to production orders (jobs or tasks and their due dates) in an optimized manners;
- The results are time allocations of production resources to different jobs (job sequences on each production resources);
- All the orders can be completed while all production resources are utilized with their loads being balanced.

Forecast of future demand

↓

Aggregate plan

↓

Master production schedule (MPS)
Schedule of production quantities by product and time period

↓

Material Requirement Planning (MRP)
Generate production orders and purchase order

↓

Operations Scheduling
To meet quantities and time requirements for MRP

Introduction-Objectives of Operations Scheduling

- **Objectives of operations scheduling**
  1) Meet due date;
  2) Minimize WIP inventory;
  3) Minimize the average flow time through the systems;
  4) Provide for high machine/worker (time) utilization (minimize idle time);
  5) Reduce setup cost;
  6) Minimize production and worker costs

Introduction-Functions of Scheduling and Control

- **The following functions must be performed in scheduling and controlling a shop floor:**
  - Allocating orders, equipments, and personnel to work centers or other specified location-Short term capacity planning;
  - Determining the sequence of orders (i. e. job priorities);
  - Initializing performance of the scheduled work, commonly termed the dispatching of jobs;
  - Shop-floor control, involving
    - Reviewing the status and controlling the progress of orders as they are being worked on;
    - Expediting the late and critical orders;
  - Revising the schedules in light of changes in order status.

Introduction-Elements of Shop Floor Scheduling Problems

- The classic approaches to shop floor scheduling focuses on the following six elements:
  - Job arrival patterns: static or dynamic
    - Static: jobs arrive in batch;
    - Dynamic: jobs arrive over time interval according to some statistical distribution.
  - Numbers and variety of machines in the shop floor
    - If there is only one machine or if a group of machines can be treated as one machine, the scheduling problem is much more simplified;
    - As number of variety of machines increase, the more complex the scheduling problems is likely to become.

Introduction-Elements of Shop Floor Scheduling Problems

- Ratio of workers to machines
  - Machine limited system: more workers than machine or equal number workers and machines;
  - Labor-limited system: more machines than worker.
- Flow pattern of jobs: flow shop or job shop
  - Flow shop: all jobs follow the same paths from one machine to the next;
  - Job shop: no similar pattern of movement of jobs from one machine to the next.

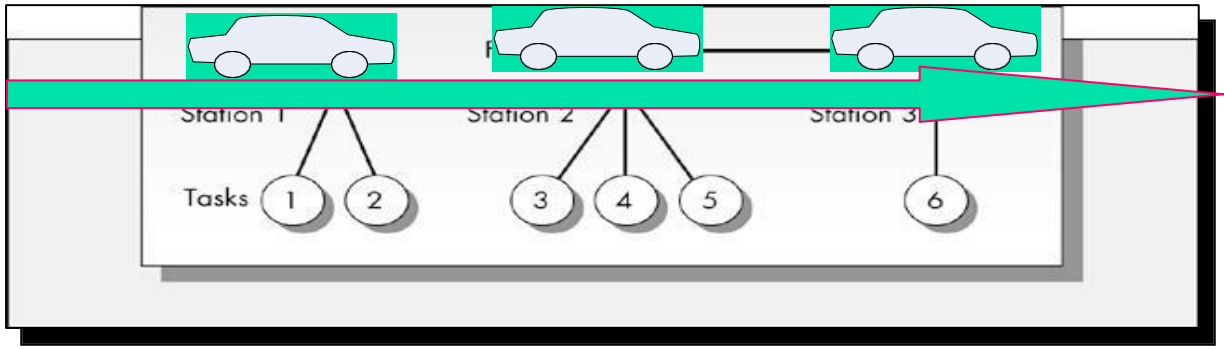
Introduction-Elements of Shop Floor Scheduling Problems

- Job sequencing
  - Sequencing or priority sequencing: the process of determining which job is started first on some machines or work center by priority rule;
  - Priority rule: the rule used for obtaining a job sequencing;
- Priority rule evaluation criteria
  - To meet corresponding objectives of scheduling;
  - Common standard measures:
    - Meeting due date of customers or downstream operations;
    - Minimizing flow time (the time a job spends in the shop flow);
    - Minimizing WIP;
    - Minimizing idle time of machines and workers (Maximizing utilization).

Introduction - Flow Shop

Flow shop:

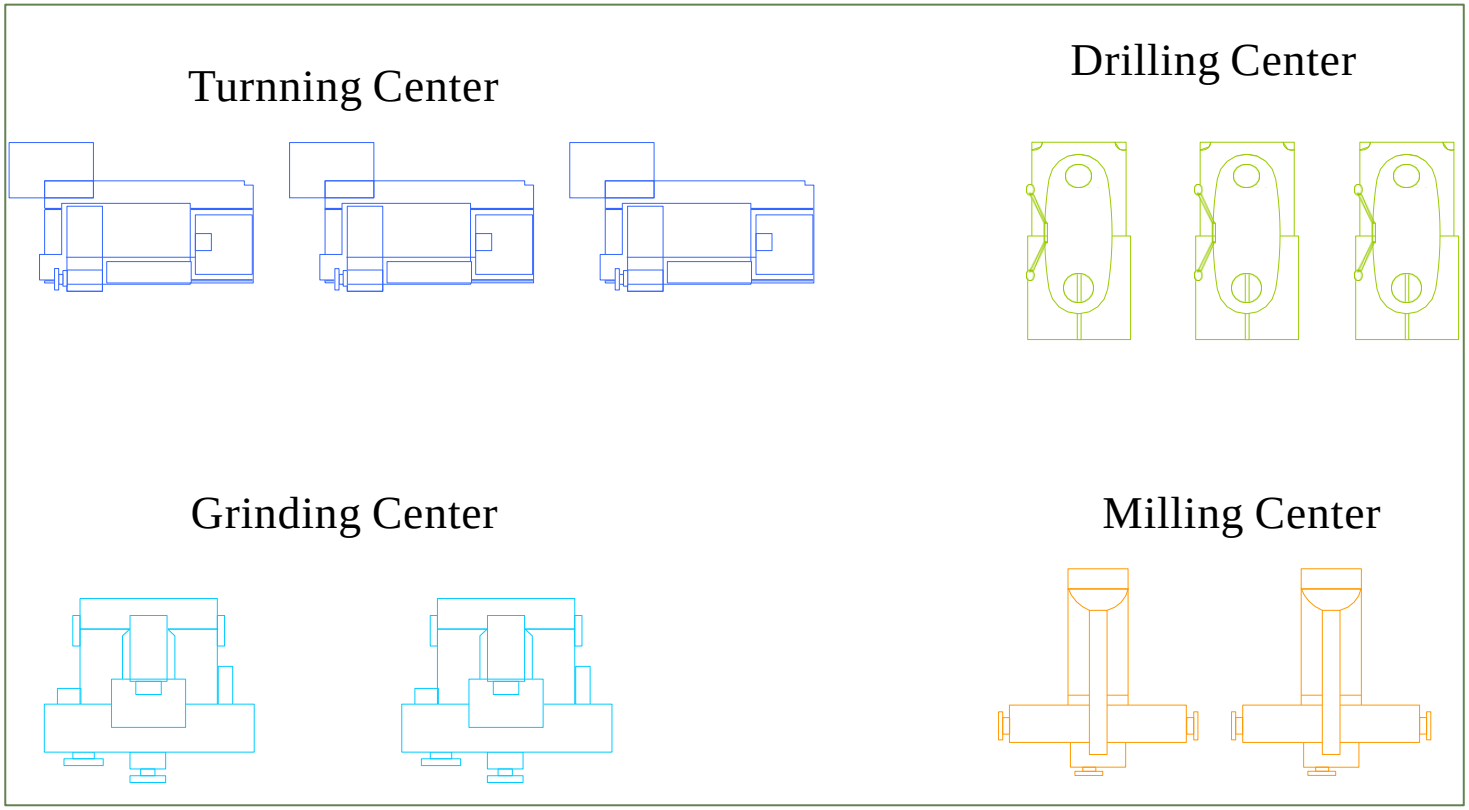
- Each of the *n* jobs must be processed through the *m* machines in the same order.
- Each job is processed exactly once on each machine.



An assembly line is a classic example of flow shop

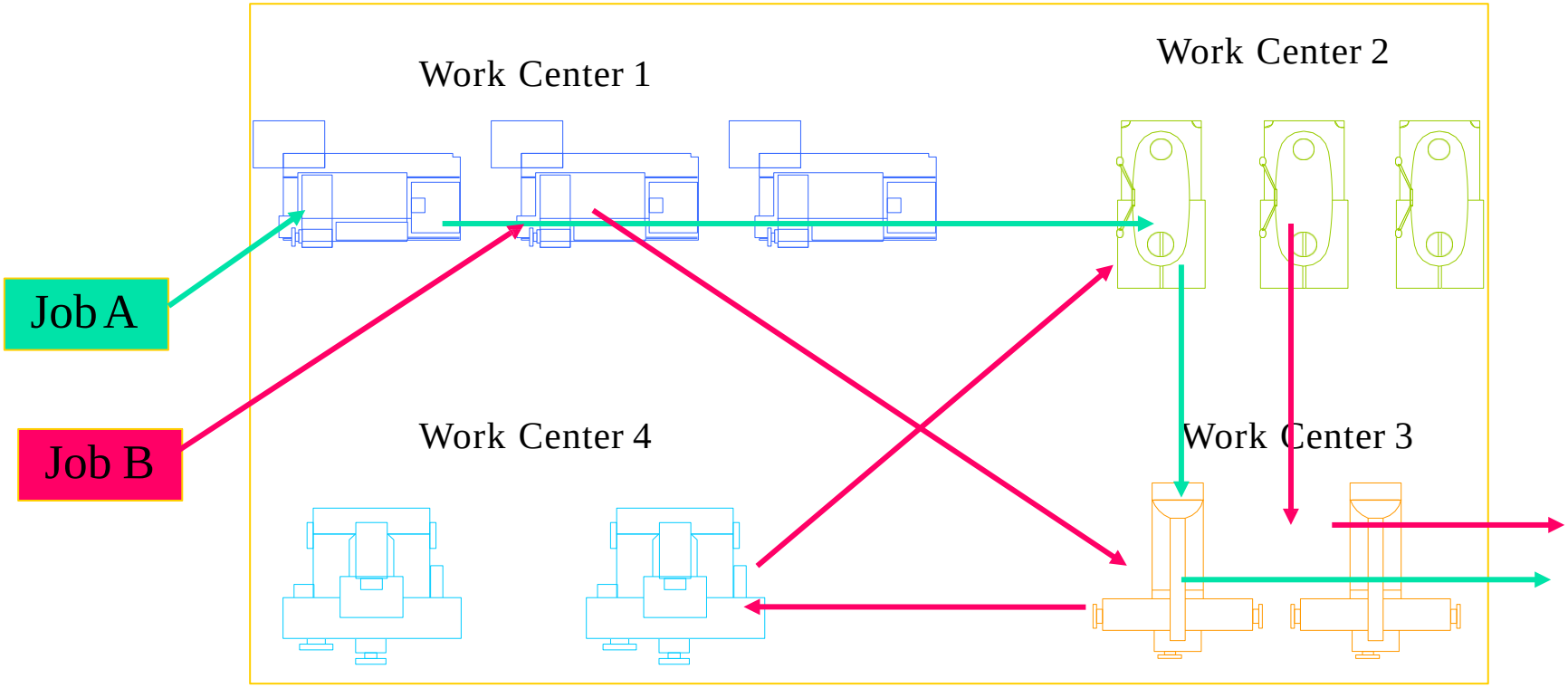
- Every cars go through all the stations one by one in the same sequences;
- Same tasks are performed on each car in each station;
- Its operations scheduling is simplified as assembly line balancing;
- An assembly balancing problem is to determine the number of stations and to allocate tasks to each station.

Introduction-Job Shop



A job shop is organized by machines which are grouped according to their functions.

Introduction-Job Shop

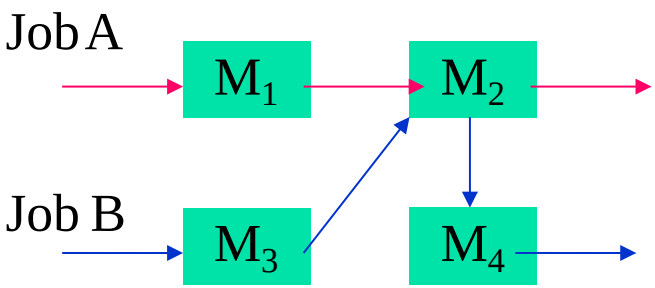


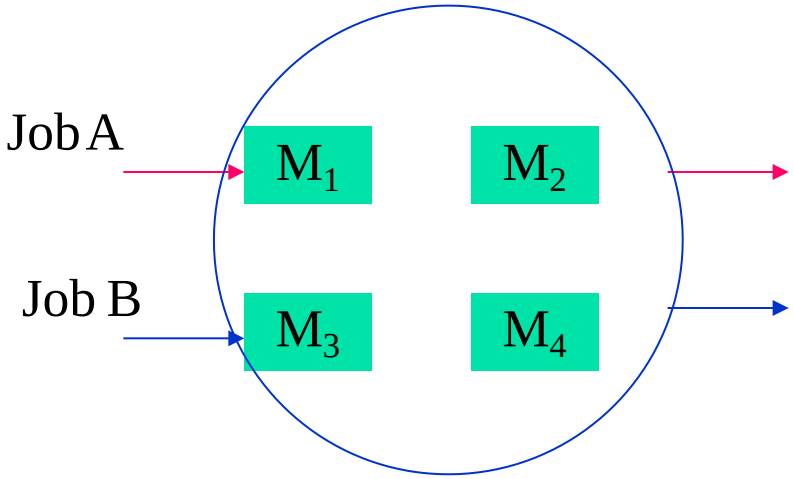
- Not all jobs are assumed to require exactly the same number of operations, and some jobs may require multiple operations on a single machine (Reentrant system, Job B twice in work center 3).
- Each job may have a different required sequencing of operations.
- No all-purpose solution algorithms for solving general job shop problems ;
- Operations scheduling of shop floor usually means job shop scheduling;

Shop Scheduling Terminology

1. Parallel processing versus sequential processing

- Sequencing Processing: the *m* machines are distinguishable, and different operations are performed by different machines.
- Parallel processing: The machines are identical, and any job can be processed on any machine.





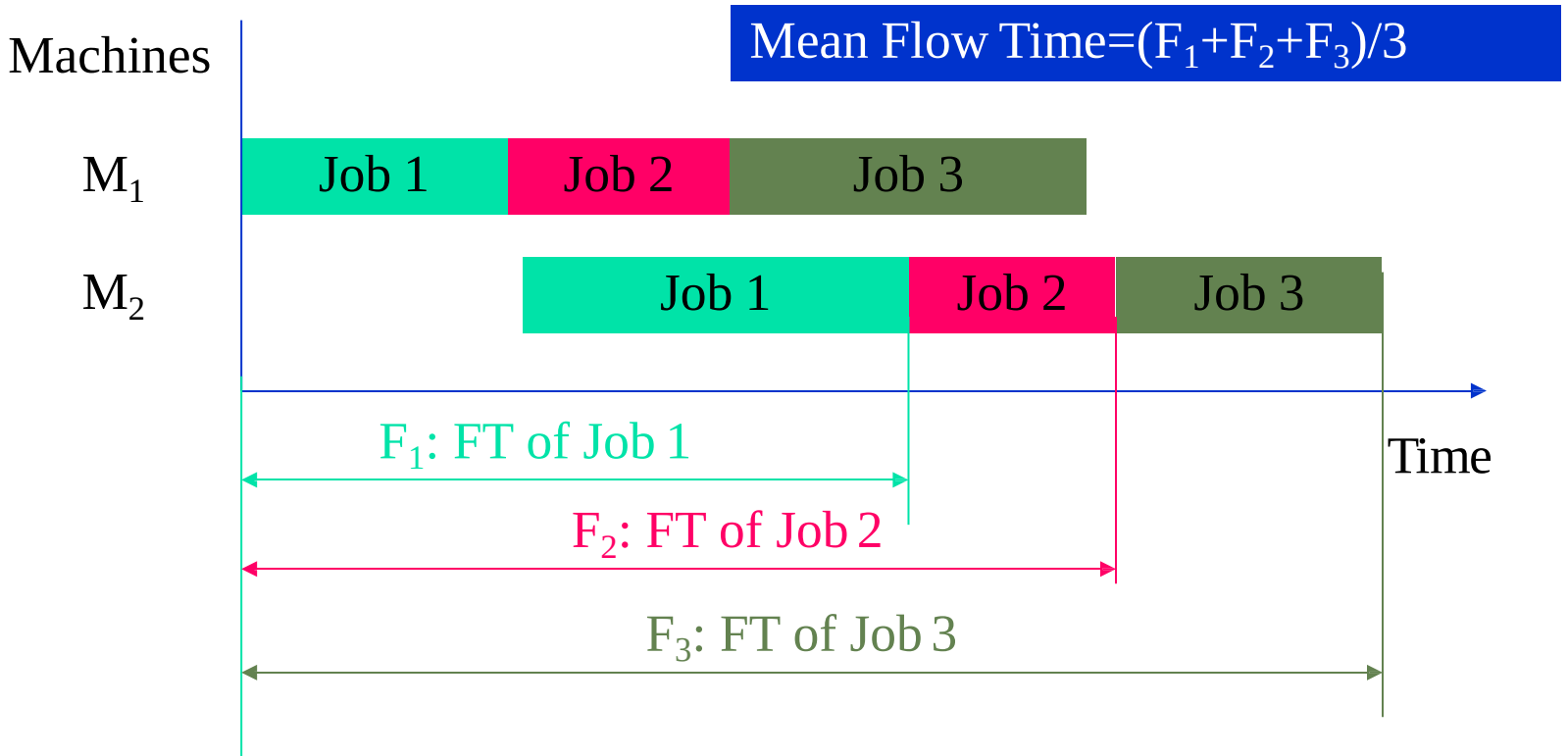
- M_1, M_2, M_3, and M_4 are different;
- Job A has 2 operations which should be processed on different Machines: M_1 and M_2;
- Job B has 3 operations which should be processed on different Machines: M_3, M_2 and M_4;

- M_1, M_2, M_3, and M_4 are identical;
- Jobs A and B can be processed on any one of the 4 machines

Shop Scheduling Terminology

2 Flow time

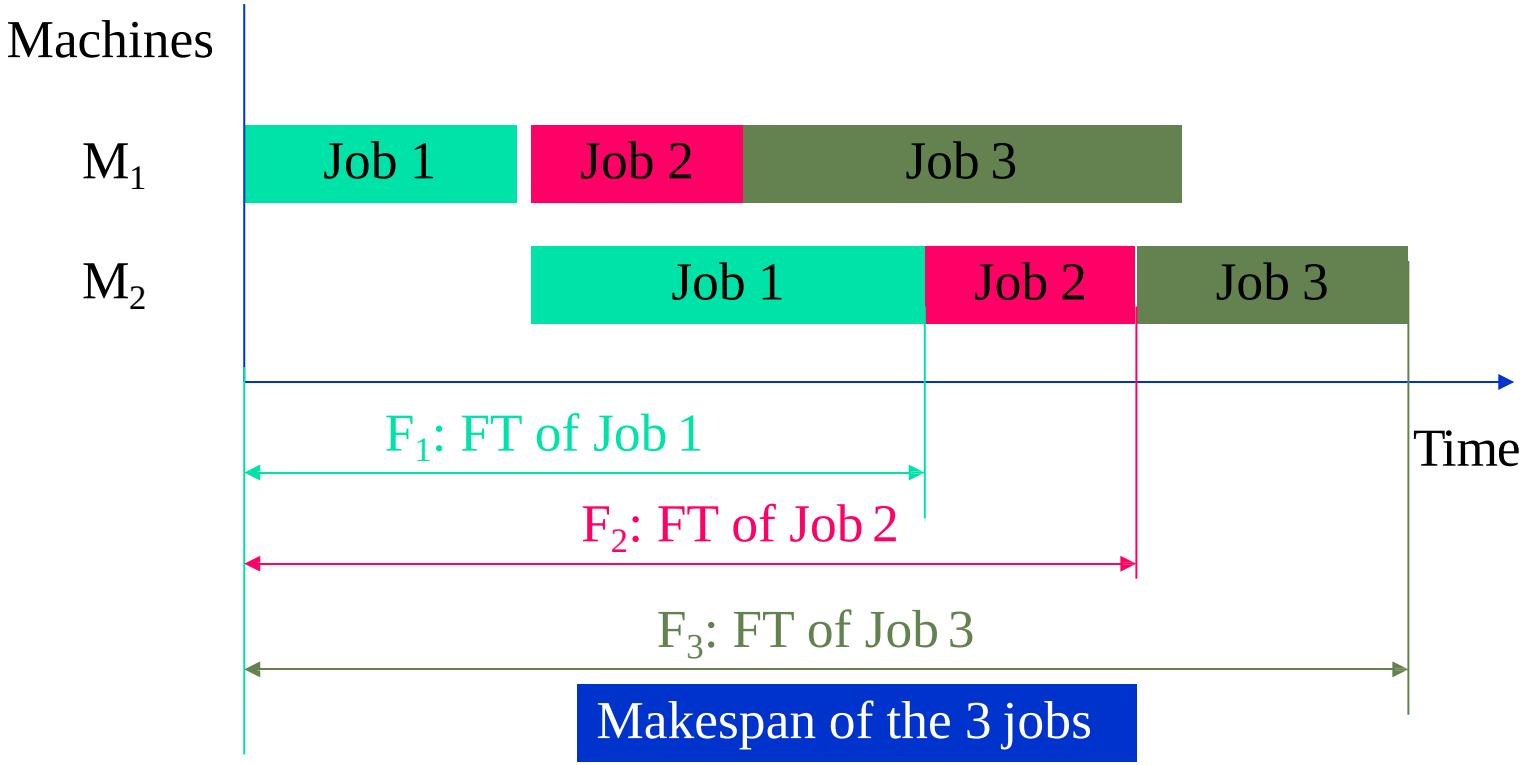
- The flow time of job *i* is the time that elapses from the initiation of that job on the first machine to the completion of job *i*.
- The mean flow time, which is a common measure of system performance, is the arithmetic average of the flow times for all *n* jobs



Shop Scheduling Terminology

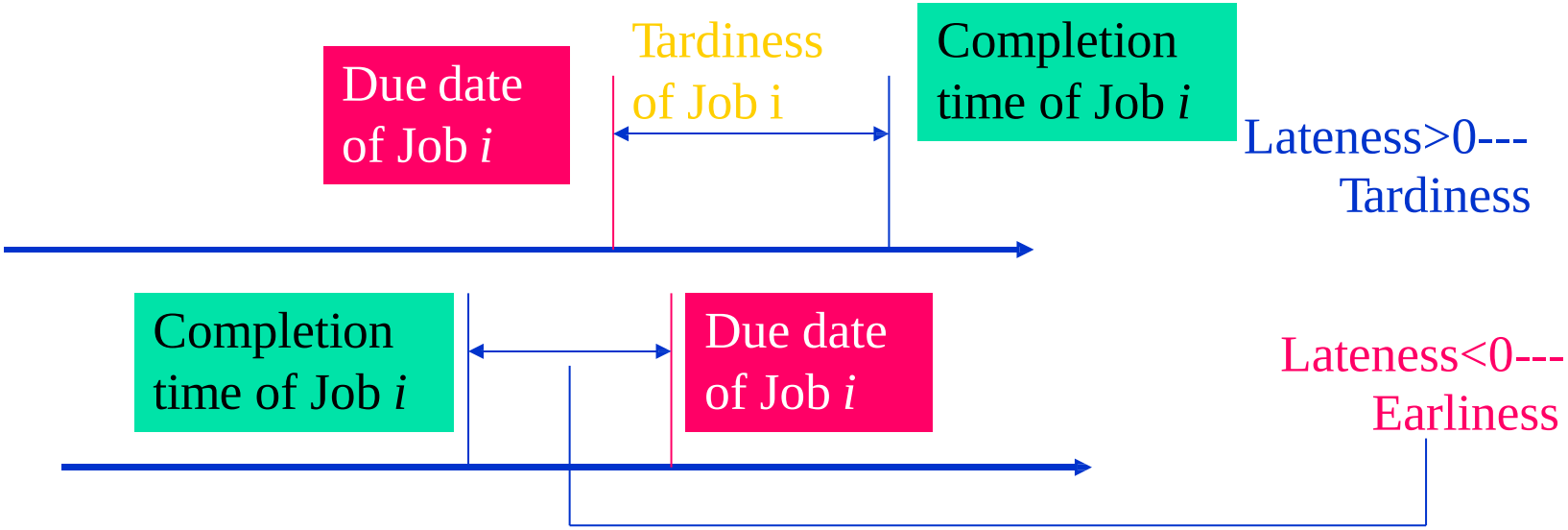
3. Makespan

- The makespan is the time required to complete a group of jobs (all *n* jobs).
- Minimizing the makespan is a common objective in multiple-machine sequencing problems.



4. Tardiness and lateness

- Tardiness is the positive difference between the completion time and the due date of a job.
- Lateness refers to the difference between the job completion time and its due date and differs from tardiness in that lateness can be either positive or negative.
- If lateness is positive, it is tardiness; when it is negative, it is earliness



When the completion of Job is earlier than due date, the tardiness is 0

Sequencing Rules

FCFS *(first come-first served)*

- Jobs are processed in the sequence in which they entered the shop;
- The simplest and nature way of sequencing as in queuing of a bank

SPT *(shortest processing time)*

- Jobs are sequenced in increasing order of their processing time;
- The job with shortest processing time is first, the one with the next shortest processing time is second, and so on;

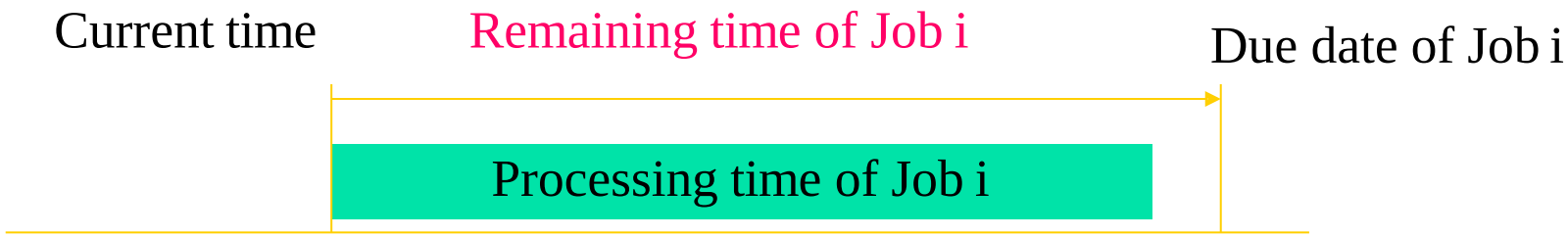
EDD *(earliest due date)*

- Jobs are sequenced in increasing order of their due dates;
- The job with earliest due date is first, the one with the next earliest due date is second, and so on;

Sequencing Rules

CR *(Critical ratio)*

- Critical ratio is the remaining time until due date divided by processing time;
- Scheduling the job with the smallest CR next;



CR_i=Remaining time of Job i/Processing time of Job i

=(Due date of Job i-current time)/Processing time of Job i

- CR provides the balance between SPT and EDD, such that the task with shorter remaining time and longer processing time takes higher priority;
- CR will become smaller as the current time approaches due date, and more priority will given to one with longer processing time;
- For a job, if the numerator of its CR is negative (the job completion time has already crossed its due date), it is naturally scheduled next;
- **If more than one jobs are tardy, higher priority is given to one that has shorter processing time (SPT).**

Example

- A machine center in a job shop for a local fabrication company has five unprocessed jobs remaining at a particular point in time. The jobs are labeled 1, 2, 3, 4, and 5 in the order that they entered the shop. The respective processing times and due dates are given in the table below.
- Sequence the 5 jobs by above 4 rules and compare results based on mean flow time, average tardiness, and number of tardy jobs

Job number	Processing Time	Due Date
1	11	61
2	29	45
3	31	31
4	1	33
5	2	32

Sequencing Rules——FCFS

Mean Flow time=268/5=53.6
Average tardiness=121/5=24.2
No. of tardy jobs=3.

Job number	Processing Time	Due Date
1	11	61
2	29	45
3	31	31
4	1	33
5	2	32

Job	Completion Time	Due Date	Tardiness
1	11	61	0
2	40	45	0
3	71	31	40
4	72	33	39
5	74	32	42
Totals	268		121

Sequencing Rules——SPT

Mean Flow time=135/5=27.0
Average tardiness=43/5=8.6
No. of tardy jobs=1.

Job number	Processing Time	Due Date
1	11	61
2	29	45
3	31	31
4	1	33
5	2	32

Job	Processing Time	Completion Time	Due Date	Tardiness
4	1	1	33	0
5	2	3	32	0
1	11	14	61	0
2	29	43	45	0
3	31	74	31	43
Totals		135		43

Sequencing Rules——EDD

Mean Flow time=235/5=47.0
Average tardiness=33/5=6.6
No. of tardy jobs=4.

Job number	Processing Time	Due Date
1	11	61
2	29	45
3	31	31
4	1	33
5	2	32

Job	Processing Time	Completion Time	Due Date	Tardiness
3	31	31	31	0
5	2	33	32	1
4	1	34	33	1
2	29	63	45	18
1	11	74	61	13
Totals		235		33

Sequencing Rules——CR

Current time: t=0

Job number	Processing Time	Due Date	Critical Ratio
1	11	61	61/11(5.545)
2	29	45	45/29(1.552)
~~3~~	~~31~~	~~31~~	~~31/31(1.000)~~
4	1	33	33/1 (33.00)
5	2	32	32/2 (16.00)

Current time should be reset after scheduling one job

Current time: t=31

Job number	Processing Time	Due Date-Current Time	Critical Ratio
1	11	30	30/11(2.727)
~~2~~	~~29~~	~~14~~	~~14/29(0.483)~~
4	1	2	2/1 (2.000)
5	2	1	1/2 (0.500)

Sequencing Rules——CR

Mean Flow time=289/5=57.8
Average tardiness=87/5=17.4

No. of tardy jobs=4.

Current time=60

Job number	Processing Time	Due Date-Current Time	Critical Ratio
1	11	1	1/11(0.0909)
~~4~~	~~1~~	~~-27~~	~~-27/1<0~~
5	2	-28	-28/2<0

Both Jobs 4 and 5 are late, however Job 4 has shorter processing time and is thus scheduled first. Finally, job 1 is scheduled at the end.

Job number	Processing Time	Completion Time	Tardiness
3	31	31	0
2	29	60	15
4	1	61	28
5	2	63	31
1	11	74	13
Totals		289	87

Sequencing Rules – Modified CR

- In the given CR procedure, when multiple remaining jobs are tardy, the one with the shortest processing time is selected to be processed next.

- In other words, when multiple critical ratios are negative, this approach implicitly considers only the **denominator** (processing time) of the negative critical ratios, and selects the job with the smallest value of the denominator (shortest processing time among tardy jobs).

- An alternative approach is to consider only the **numerator** (due date – current time) and the job with the smaller numerator is selected as the next job for processing.

- This modified CR rule helps reduce the maximum tardiness of jobs, compared to the original CR rule.

Sequencing Rules – Modified CR

Current time: t=0

Job number	Processing Time	Due Date	Critical Ratio
1	11	61	61/11(5.545)
2	29	45	45/29(1.552)
~~3~~	~~31~~	~~31~~	~~31/31(1.000)~~
4	1	33	33/1 (33.00)
5	2	32	32/2 (16.00)

Current time should be reset after scheduling one job

Current time: t=31

Job number	Processing Time	Due Date-Current Time	Critical Ratio
1	11	30	30/11(2.727)
~~2~~	~~29~~	~~14~~	~~14/29(0.483)~~
4	1	2	2/1 (2.000)
5	2	1	1/2 (0.500)

Sequencing Rules – Modified CR

Mean Flow time=290/5=58
Average tardiness=88/5=17.6

No. of tardy jobs=4.

Current time=60

Job number	Processing Time	Due Date-Current Time	Critical Ratio
1	11	1	1/11(0.0909)
4	1	-27	-27/1<0
~~5~~	~~2~~	~~-28~~	~~-28/2<0~~

Both Jobs 4 and 5 are late, however the CR of Job 5 has the smaller numerator and is thus scheduled first. Finally, job 1 is scheduled at the end.

Job number	Processing Time	Completion Time	Tardiness
3	31	31	0
2	29	60	15
5	2	62	30
4	1	63	30
1	11	74	13
Totals		290	88

Sequencing Rules——Summary

Rule	Mean Flow Time	Mean Tardiness	Maximum Tardiness	Number of Tardy Jobs
FCFS	53.6	24.2	42	3
SPT	27.0	8.6	43	1
EDD	47.0	6.6	18	4
CR	57.8	17.4	31	4
Modified CR	58	17.6	30	4

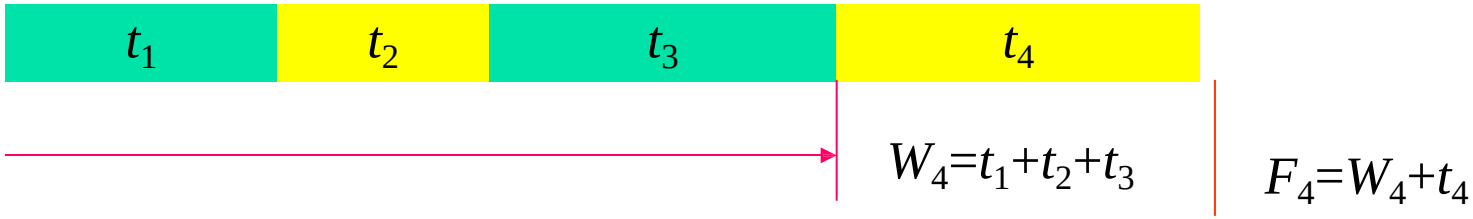
Discussions

- SPT minimizes the mean flow time;
- EDD minimizes the maximum tardiness;
- Always true? Yes!
- Mean tardiness … ? – No polynomial time algorithms

Sequencing Theory for A Single Machine

Assuming that n jobs are to be processed through one machine. For each job i, define the following quantities:

- t_i=Processing time for job i, constant for job i;
- d_i=Due date for job i, constant for job i;
- W_i=Waiting time for job i, the amount of time that the job must wait before its processing can begin.
  - When all the jobs are processed continuously, W_i is the sum of the processing times for all of the preceding jobs;



- F_i=Flow time for job i, the waiting time plus the processing time: $F_i= W_i+ t_i$;
- L_i=Lateness of job i , $L_i= F_i- d_i$, either positive or negative;
  - T_i=Tardiness of job i, the positive part of L_i, T_i=max[L_i,0] ;
  - E_i=Earliness of job i, the negative part of L_i, E_i =max[-L_i,0]

Sequencing Theory for A Single Machine

- Maximum Tardiness $T_{\max} = \max\{T_1, T_2, \ldots, T_n\}$
- Mean Flow Time $\overline{F}' = \frac{1}{n}\sum_{i=1}^{n} F_i$
- Suppose that 4 jobs J_1, J_2, J_3, J_4 need to be scheduled

- For example a schedule is J_3-J_2-J_1-J_4

- Considered as a permutation of integers 1, 2, 3, 4: 3, 2, 1, 4.

- For only a single machine, every schedule can be represented by a permutation (ordering) of the integers 1, 2, 3, …, n.
- There are totally n! (the factorial of n) different permutations.

- A permutation of integers 1, 2, …, n is expressed by [1], [2], …, [n], which represents a schedule;
- [i] denotes the integer that put in the ith place in the permutation;
- In case of a schedule 3, 2, 1, 4, [1]=3, [2]=2, [3]=1, and [4]=4;

Sequencing Theory for A Single Machine

1.Shortest-Processing-Time Scheduling

Theorem The scheduling rule that minimizes the mean flow time *F'* is SPT.

■Suppose a schedule is [1], [2], … [*k*], [*k*+1], … [*n*], the flow time of the job that is scheduled in position *k* is given by, say job in position 3:

$$F_{[k]} = \sum_{i=1}^{k} t_{[i]}$$

$t_{[1]}$ (t_2)	$t_{[2]}$ (t_1)	$t_{[3]}$ (t_4)	$t_{[4]}$ (t_3)

$F_{[2]}=t_{[1]}+t_{[2]}=t_2+t_1$

■The mean flow time of all jobs on the schedule is given by

$$F' = \frac{1}{n}\sum_{i=1}^{n} F_{[k]} = \frac{1}{n}\sum_{k=1}^{n}\sum_{i=1}^{k} t_{[i]}$$

Sequencing Theory for A Single Machine

1.Shortest-Processing-Time Scheduling

Theorem The scheduling rule that minimizes the mean flow time *F'* is SPT

■The mean flow time is given by

$$F' = \frac{1}{n}\sum_{i=1}^{n} F_{[k]} = \frac{1}{n}\sum_{k=1}^{n}\sum_{i=1}^{k} t_{[i]}$$

■The double summation term may be written in a different form. Expanding the double summation, we obtain

k=1:$t_{[1]}$;

k=2:$t_{[1]}$+ $t_{[2]}$;

…;

k=n:$t_{[1]}$+ $t_{[2}$+…$t_{[n]}$

■By summing down the column rather than across the row, we may rewrite *F'* in the form

$nt_{[1]}+(n-1)t_{[2]}+\ldots+t_{[n]}$

■Clearly, it is minimized by setting

$$t_{[1]} \le t_{[2]} \le \ldots \le t_{[n]}$$

SPT sequencing rule: the job with shortest processing time *t* is set first

Sequencing Theory for A Single Machine

1. **Shortest-Processing-Time Scheduling (Cont.)**

Corollary The following measures are equivalent:

- Mean flow time
- Mean waiting time
- Mean lateness

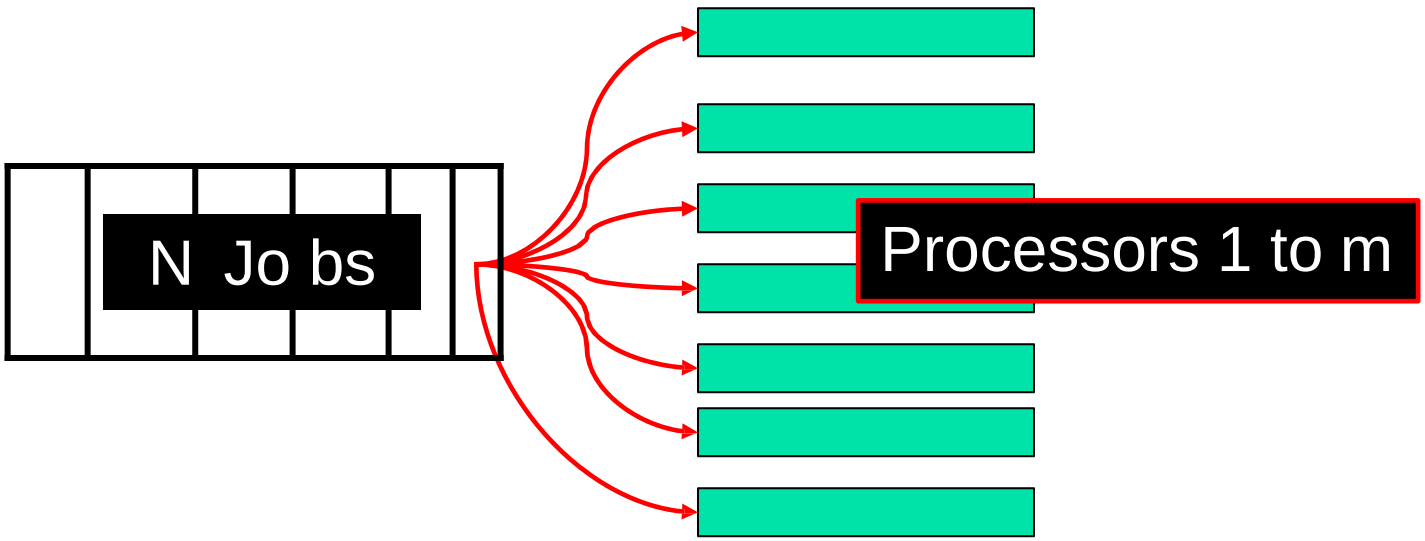
SPT minimizes mean flow time, mean waiting time, and mean lateness for single machine sequencing.

2. **Earliest-Due-Date Scheduling**: If the objective is to minimize the maximum lateness, then the jobs should be sequenced according to their due dates. That is, $d_{[1]} \leq d_{[2]} \leq \ldots d_{[n]}$.

Some Issues:

- **All of these algorithms ignore Constraints that may be desired/required**
  - Valued Customers orders
  - Promises to others
  - Rush Jobs
  - Rework/Scrap, etc.
- **These issues must be considered and the rest of jobs must be scheduled around them**

Parallel Machine Problem:



Assumptions:

1. any job can be processed on any machine.
2. process time is identical on each machine (i.e. they are identical machines).
3. once a job is processed on a machine, it is completed.

Algorithms to Minimize Mean Flow Time and/or Makespan:

- **One For Minimizing Mean Flow on m parallel processors:**
  - Step 1: Sequence all tasks in SPT order.
  - Step 2: Take Jobs from list and assign to the processor with **least amount** of assigned time. Break ties arbitrarily.
  - Step 3: Repeat for all jobs.

- **One to Minimize Makespan while *controlling* Mean Flow on m parallel processors :**
  - Step 1: Sequence the n tasks in Longest Processing time (LPT) order.
  - Step 2: Take tasks from list and assign to the processor with **least amount** of assigned time. Break Ties arbitrarily.
  - Step 3: After tasks are assigned, reverse their sequence on each machine, putting the tasks on each processor in SPT order.

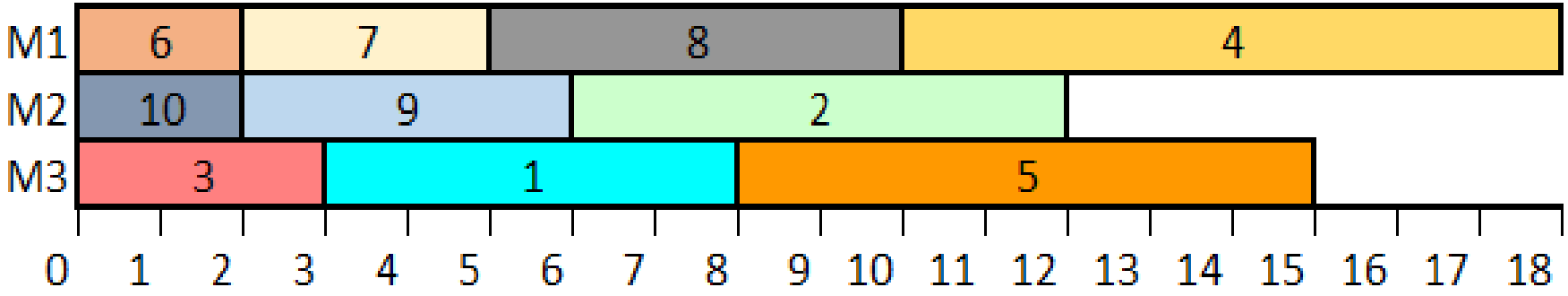
Example

Given Order		SPT Order		LPT Order	
Job #	Pr. Time	Job #	Pr. Time	Job #	Pr. Time
1	5	6	2	4	8
2	6	10	2	5	7
3	3	3	3	2	6
4	8	7	3	1	5
5	7	9	4	8	5
6	2	1	5	9	4
7	3	8	5	3	3
8	5	2	6	7	3
9	4	5	7	6	2
10	2	4	8	10	2

SPT Algorithm

Job #	Pr. Time	M1	M2	M3
6	2	#6 - 2		
10	2		#10 - 2	
3	3			#3 - 3
7	3	#7 - 5		
9	4		#9 - 6	
1	5			#9 - 8
8	5	#8 - 10		
2	6		#2 - 12	
5	7			#5 - 15
4	8	#4 - 18		

Presented as Gantt Chart:



Makespan (length of schedule) = **18 days**

Mean Flow Time = $\frac{\text{Sum of Completion Times}}{\text{Number of Jobs}} = \frac{81}{10} = \mathbf{8.1\ days}$

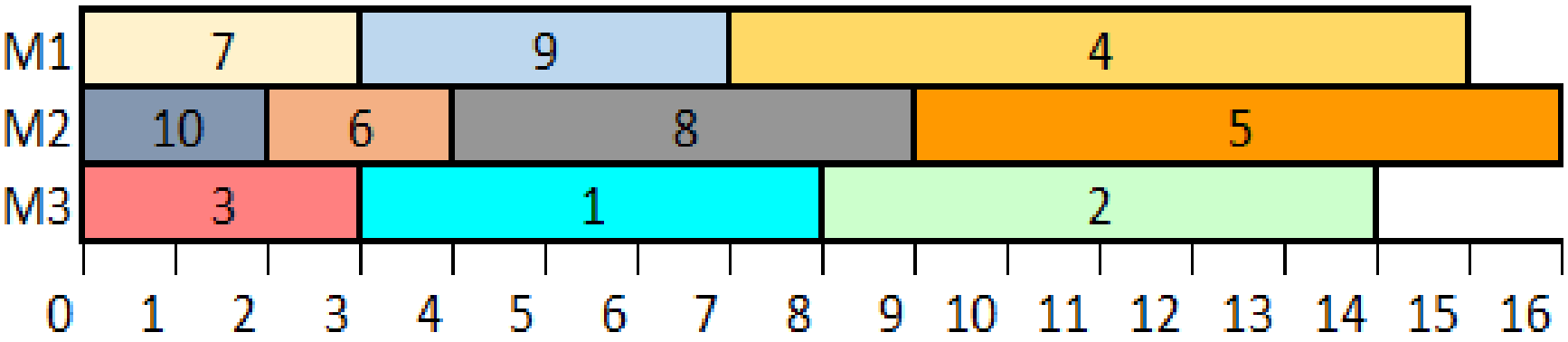
Mean Machine Idle Time = $\frac{\text{Sum of Idle Times}}{\text{Number of Machines}} = \frac{0+6+3}{3} = \mathbf{3\ days}$

LPT Approach:

Job #	Pr. Time	M1	M2	M3
4	8	#4 - 8		
5	7		#5 - 7	
2	6			#2 - 6
1	5			#1 - 11
8	5		#8 - 12	
9	4	#9 - 12		
3	3			#3 - 14
7	3	#7 - 15		
6	2		#6 - 14	
10	2		#10 - 16	

To schedule: **REVERSE ORDER** on each machine

Presented as Gantt Chart:



Makespan = **16 days**

Mean Flow Time = $\frac{\text{Sum of Completion Times}}{\text{Number of Jobs}} = \frac{81}{10} = \mathbf{8.1\ days}$

Mean Machine Idle Time = $\frac{\text{Sum of Idle Times}}{\text{Number of Machines}} = \frac{1+0+2}{3} = \mathbf{1\ day}$

Considering Lateness

- **To minimize Maximum Tardiness**
  - Step 1: Sequence task by EDD
  - Step 2: Take tasks from EDD order list then schedule them on processor with least assigned task time. Break ties arbitrarily.
- **To reduce Mean Tardiness,**
  - Make up several schedules using SPT, LPT, EDD, Slack-based rule (Slack = Due Date – Processing Time).
  - Select the sequence with the minimum mean tardy time.

Serial Machine Problem – Flow Shop Scheduling

- **In the 2-machine problem with 'n' different products, we have potentially: $(n!)^2$ different schedules – therefore, full enumeration is (usually) impossible.**
- **Rules used in making decisions:**
  - Minimizing Makespan (time from start of 1st job on M1 to completion of last job on M2)
  - Minimizing Mean Flow time (Equivalent to minimizing mean waiting time or mean resident time)
  - Minimize Mean Idle Time
  - Minimize measures of Tardiness

Johnson's Rule for two-machine flow shops

- Let P_{ik} be the time required to process job i on machine k.

- Consider two jobs i and j in a two-machine flow shop.

- Johnson's Rule states that
  - "**In an optimal sequence, job $\boldsymbol{i}$ precedes job $\boldsymbol{j}$, if $\min\{P_{i1}, P_{j2}\} \leq \min\{P_{j1}, P_{i2}\}$.**"

- Johnson's Rule is usually implemented in the form of an algorithm that is optimal by construction.

Algorithm to implement Johnson's Rule

- **Step 1:** Find the least processing time among unscheduled jobs.

- **Step 2a:** If the shortest time in Step 1 occurs on machine M1, select the corresponding job and place it in the first available position in the sequence.

- **Step 2b:** Similarly, if this shortest time occurs on machine M2, place the corresponding job in the last available position in the sequence.

- **Step 3:** Remove the selected job and repeat Step 1 until all jobs have been sequenced.

Proof of Johnson's Rule:

- Let N be the set of jobs. We define $N^+ = \{J_i \mid p_i \leq q_i, J_i \in N\}$ and $N^- = \{J_i \mid p_i > q_i, J_i \in N\}$, where p_i and q_i are the processing times of job i on machines 1 and 2, respectively.

- Let S be an optimal schedule different from the schedule produced by Johnson's rule.

- Let J_i, J_j be the first pair of jobs in schedule S such that the relation $J_i \preccurlyeq_J J_j$ holds but J_j precedes J_i .

- Here, $(J_{j_1}, J_{j_2}, \ldots, J_{j_k})$ is the sequence of jobs scheduled between J_j and J_i . We show that eliminating the out-of-order jobs between J_i and J_j will not increase the makespan.

- Moreover, we need to guarantee that no two jobs that are originally in order will become out of order.

We consider the following three cases:

- Case 1: J_j , $J_i \in N^+$: Because $J_j \preccurlyeq_J J_{j'}$, for $j_1 \le j' \le j_k$, by transitivity, $J_i \preccurlyeq_J J_j \preccurlyeq_J J_{j'}$, for $j_1 \le j' \le j_k$. More pairs of out-of-order jobs can possibly be eliminated and no extra pair of jobs become out of order.

- Case 2: J_j , $J_i \in N^-$: Transforming the original sequence (J_j , J_{j_1} , . . . , J_{j_k} , J_i) into (J_{j_1} , . . . , J_{j_k} , J_i , J_j) makes jobs J_i and J_j in order without causing a new pair of out-of-order jobs.

- Case 3: $J_j \in N^-$, $J_i \in N^+$: The reasoning for the new subsequence (J_i , J_j , J_{j_1} , . . . , J_{j_k}) (respectively, subsequence (J_{j_1} , . . . , J_{j_k} , J_i , J_j)) is the same as that used for proving Case 1 (respectively, Case 2).

- Continuing the above swapping procedure, if necessary, we will finally come up with a schedule as specified by Johnson's algorithm. The proof is completed.

Lets Try an Example:

	M1	M2
J1	5	2
J2	1	6
J3	9	7
J4	3	8
J5	10	4

The shortest processing time corresponds to that of job 2 on machine 1. Select J2 and place it in the first position in the sequence. Remove J2 from consideration.

Continuing: (sequence is: 2-_- _- _- _)

	M1	M2
J1	5	**2**
J2	1	6
J3	9	7
J4	3	8
J5	10	4

Among remaining jobs, the minimum time belongs to job 1 on machine 2. Select J1 and place it in the last position in the sequence. Remove J1 from consideration.

Continuing: (sequence is 2-_- _- _-1)

	M1	M2
J1	5	2
J2	1	6
J3	9	7
J4	**3**	8
J5	10	4

The next least time belongs to job 4 on machine 1. Select J4 and place it in the second position (first position is already taken) in the sequence. Remove J4 from consideration.

Continuing: (sequence is 2-4-_ _-1)

	M1	M2
J1	5	2
J2	1	6
J3	9	7
J4	3	8
J5	10	4

The next least time belongs to job 5 on machine 2. Select J5 and place it in the second-last position (last position is already taken) in the sequence. Remove J4 from consideration.

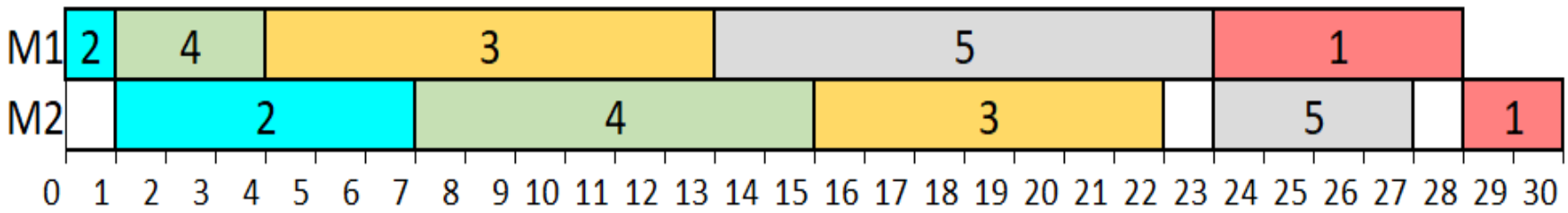
Continuing: (sequence is 2-4-_-5-1)

	M1	M2
J1	5	2
J2	1	6
J3	9	7
J4	3	8
J5	10	4

Finally, schedule the only remaining job (J3) in the only remaining position (third) in the sequence. Since no jobs remain unscheduled, the algorithm terminates.

Summarizing:

The final sequence based on Johnson's Algorithm is 2-4-3-5-1.



Makespan = **30 units**

Mean Flow Time = $\frac{\text{Sum of Completion Times}}{\text{Number of Jobs}} = \frac{7+15+22+27+30}{5} = \mathbf{20.2}$ **units**

An SPT-based Heuristic to minimize Mean Flow Time in 2-machine flow shops

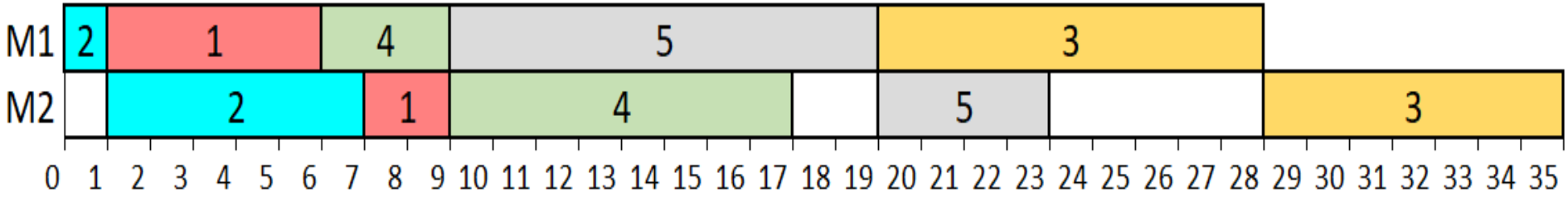
Step 1: Find the total processing time (across both machines) for each job.

Step 2: Arrange the jobs in non-decreasing order of their total processing times, with ties broken in favour of the job having shorter processing time on M1.

	M1	M2	Total
J1	5	2	7
J2	1	6	7
J3	9	7	16
J4	3	8	11
J5	10	4	14

Summarizing:

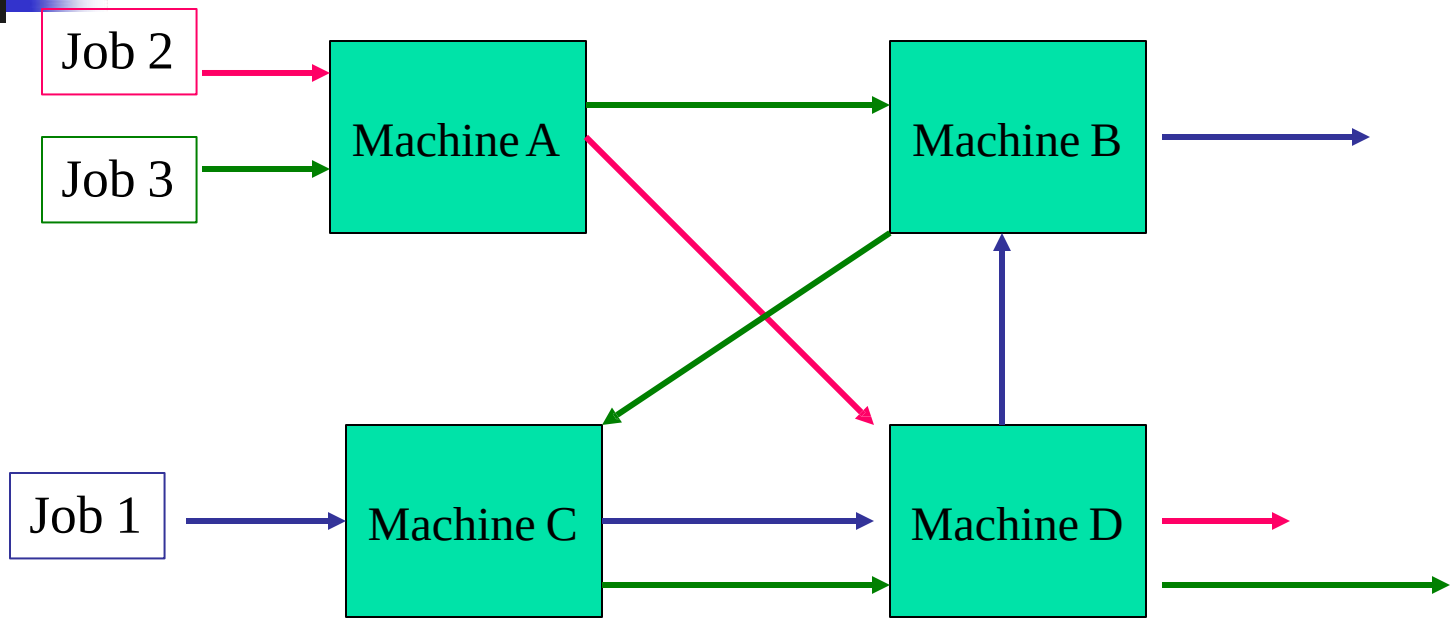
Based on the heuristic, the final sequence is 2-1-4-5-3.



Makespan = **35 units**

Mean Flow Time = $\frac{\text{Sum of Completion Times}}{\text{Number of Jobs}} = \frac{7+9+17+23+35}{5} = \mathbf{18.2}$ **units**

Job Shops



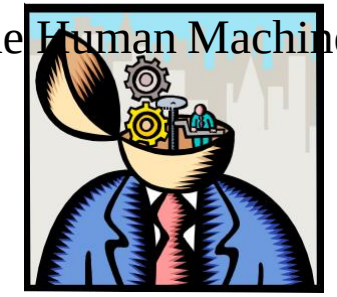
$(N!)^M$ Sequences

Dispatching Rules

Schedule an operation as soon as possible. If more than one job is waiting to be processed at the same machine, schedule the one with the best priority.

1. **SPT**
2. **FCFS**
3. **EDD**
4. **SLK (slack)**
5. **CR (critical ratio)**

The Human Machine



MILP Models for Scheduling

General Assumptions:

- All jobs are available for processing at time zero and all machines are continuously available.
- Each machine can process only one operation at a time.
- Pre-emption is disallowed, i.e., an operation cannot be interrupted while it is being processed.
- Job descriptors are known well in advance.
- The setup times for operations are independent of the processing sequence and are included in the processing time.

Relative Precedence-based MILP Model for Single Machine Scheduling

Notations:

- i, h : indices of job, $i, h \in J = \{1,2, \dots, n\}$
- n : number of jobs
- P_i : the processing time of job i
- D_i : due date of job i
- M : A large constant

Relative Precedence-based MILP Model for Single Machine Scheduling

Decision Variables:

- C_i : completion time of job i
- T_i : tardiness of job i
- X_{hi} : sequencing variable that indicates relative job precedence in the processing sequence

$$X_{hi} = \begin{cases} 1, & if\ job\ h\ precedes\ job\ i\ in\ the\ processing\ sequence \\ 0, & otherwise \end{cases}$$

Relative Precedence-based MILP Model for Single Machine Scheduling – Mean Tardiness Objective

Objective:

$$Min\ Z = \frac{1}{n}\sum_{i=1}^{n} T_i \qquad (1)$$

subject to

$$C_i \geq P_i;\ \forall\ i \in J \qquad (2)$$

$$C_i \geq C_h + P_i - M(1 - X_{hi});\ \forall\ \ h, i \in J : h < i \qquad (3)$$

$$C_h \geq C_i + P_h - MX_{hi};\ \forall\ \ h, i \in J : h < i \qquad (4)$$

$$T_i \geq C_i - D_i;\ \forall\ \ i \in J \qquad (5)$$

$$T_i \geq 0;\ \forall\ \ i \in J \qquad (6)$$

$$C_i \geq 0;\ \forall\ \ i \in J \qquad (7)$$

$$X_{hi} \in \{0,1\};\ \forall\ \ h, i \in J : h < i \qquad (8)$$

Relative Precedence-based MILP Model for Single Machine Scheduling – Mean Tardiness Objective

- Equation (1) represents the objective: mean tardiness.
- Constraint set (2) ensures that the first job in the sequence cannot begin processing before time zero.
- Constraint sets (3) and (4) ensure that the processing intervals of any pair of jobs do not overlap.

Relative Precedence-based MILP Model for Single Machine Scheduling – Mean Tardiness Objective

- Constraint set (5) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates.

- Constraint sets (6) and (7) ensure that the tardiness and completion times are non-negative.

- Finally, constraint set (8) imposes a binary restriction on the sequencing variables.

Example with MILP Solution

Job number	Processing Time	Due Date
1	11	61
2	29	45
3	31	31
4	1	33
5	2	32

Solution: X_{hi} **values**

X_{hi}	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$h = 1$	-	1	0	0	0
$h = 2$	-	-	0	0	0
$h = 3$	-	-	-	1	1
$h = 4$	-	-	-	-	1
$h = 5$	-	-	-	-	-

Example with MILP Solution

Note: In the given mathematical formulation for single machine scheduling, the X_{hi} variables are defined only for $h < i$.

However, after solving the model, interpreting the processing sequence only from those X_{hi} values may appear somewhat difficult.

To facilitate this interpretation, we introduce the expression $\boldsymbol{X_{ih} = 1 - X_{hi}}$. This is based on the idea that if job h precedes job i, then job i must succeed job h and vice versa.

Example with MILP Solution

Using above expression, we compute X_{ih} for all $i < h$ and populate the table as follows:

X_{hi}	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	Sum
$h = 1$	-	1	0	0	0	1
$h = 2$	0	-	0	0	0	0
$h = 3$	1	1	-	1	1	4
$h = 4$	1	1	0	-	1	3
$h = 5$	1	1	0	0	-	2

To determine the processing sequence, find the sum of entries for each row and order the jobs in decreasing order of their respective sum values.

Example with MILP Solution

From the X_{hi} values, the job sequence is found to be 3-4-5-1-2. The C_i and T_i values are:

Job i	Processing Time, P_i	Completion Time, C_i	Due Date, D_i	Tardiness, T_i
3	31	31	31	0
4	1	32	33	0
5	2	34	32	2
1	11	45	61	0
2	29	74	45	29
			Total	31
			Mean	6.2

Position-based MILP Model for Single Machine Scheduling

Notations:

- i : index of job, $i \in J = \{1,2,\dots,n\}$
- j : index of position, $j \in J = \{1,2,\dots,n\}$
- n : number of jobs
- P_i : the processing time of job i
- D_i : due date of job i

Decision Variables:

- C_j : completion time of job scheduled in the jth position
- T_j : tardiness of job scheduled in the jth position
- Y_{ij} : sequencing variable that indicates the position of job i in the processing sequence

$$Y_{ij} = \begin{cases} 1, & if\ job\ i\ is\ processed\ in\ the\ j^{th}\ position\ in\ the\ sequence \\ 0, & otherwise \end{cases}$$

Position-based MILP Model for Single Machine Scheduling – Mean Tardiness Objective

Objective:

$$Min\ Z = \frac{1}{n}\sum_{j=1}^{n} T_j \qquad (1)$$

subject to

$$C_j = \sum_{j'=1}^{j}\sum_{i=1}^{n} Y_{ij'}P_i\ ;\ \forall\ j \in J \qquad (2)$$

$$\sum_{i=1}^{n} Y_{ij} = 1;\ \forall\ j \in J \qquad (3)$$

$$\sum_{j=1}^{n} Y_{ij} = 1;\ \forall\ i \in J \qquad (4)$$

$$T_j \geq C_j - \sum_{i=1}^{n} Y_{ij}D_i\ ;\ \forall\ j \in J \qquad (5)$$

$$T_j \geq 0;\ \forall\ j \in J \qquad (6)$$

$$C_j \geq 0;\ \forall\ j \in J \qquad (7)$$

$$Y_{ij} \in \{0,1\};\ \forall\ i,j \in J \qquad (8)$$

Position-based MILP Model for Single Machine Scheduling – Mean Tardiness Objective

- Equation (1) represents the objective: mean tardiness.

- Constraint set (2) maps the completion time of a job in any position j in the sequence to the sum of processing times of the jobs in the first j positions.

- Constraint sets (3) and (4) ensure that exactly one job is assigned to every position and exactly one position is assigned to every job.

Position-based MILP Model for Single Machine Scheduling – Mean Tardiness Objective

- Constraint set (5) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates.

- Constraint sets (6) and (7) ensure that the tardiness and completion times are non-negative.

- Finally, constraint set (8) imposes a binary restriction on the sequencing variables.

Example with MILP Solution

Job number	Processing Time	Due Date
1	11	61
2	29	45
3	31	31
4	1	33
5	2	32

<u>Solution:</u> Y_{ij} values

Y_{ij}	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$
$i = 1$	0	0	0	1	0
$i = 2$	0	0	0	0	1
$i = 3$	1	0	0	0	0
$i = 4$	0	1	0	0	0
$i = 5$	0	0	1	0	0

Example with MILP Solution

From the Y_{ij} values, the job sequence is found to be 3-4-5-1-2. The C_j and T_j values are:

Job i	Position j	Processing Time, P_i	Completion Time, C_j	Due Date, D_i	Tardiness, T_j
3	1	31	31	31	0
4	2	1	32	33	0
5	3	2	34	32	2
1	4	11	45	61	0
2	5	29	74	45	29
				Total	31
				Mean	6.2

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Identical Machines)

Notations:

- i, h : indices of job, $i, h \in J = \{1,2, \dots, n\}$
- k : index of machine, $k \in K = \{1,2, \dots, m\}$
- n : number of jobs
- m : number of machines
- P_i : the processing time of job i
- D_i : due date of job i
- M : A large constant

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Identical Machines)

Decision Variables:

- C_i : completion time of job i
- T_i : tardiness of job i
- C_{max} : maximum job completion time (makespan)
- Y_{ik} : assignment variable that takes the value 1, if job i is assigned to machine k, and 0, otherwise
- X_{hi} : sequencing variable that indicates relative job precedence in the processing sequence

$$X_{hi} = \begin{cases} 1, & if\ job\ h\ precedes\ job\ i\ in\ the\ processing\ sequence \\ 0, & otherwise \end{cases}$$

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Identical Machines) – Mean Tardiness Objective

Objective:

$$Min\ Z = \frac{1}{n}\sum_{i=1}^{n} T_i \tag{1}$$

subject to

$$C_i \geq P_i;\ \forall\ i \in J \tag{2}$$

$$C_i \geq C_h + P_i - M(3 - X_{hi} - Y_{hk} - Y_{ik});\ \forall\ k \in K, ; h, i \in J : h < i \tag{3}$$

$$C_h \geq C_i + P_h - M(2 + X_{hi} - Y_{hk} - Y_{ik});\ \forall\ k \in K, ; h, i \in J : h < i \tag{4}$$

$$\sum_{k=1}^{m} Y_{ik} = 1;\ \forall\ \ i \in J \tag{5}$$

$$T_i \geq C_i - D_i;\ \forall\ \ i \in J \tag{6}$$

$$T_i \geq 0;\ \forall\ \ i \in J \tag{7}$$

$$C_i \geq 0;\ \forall\ \ i \in J \tag{8}$$

$$X_{hi} \in \{0,1\};\ \forall\ \ h, i \in J : h < i \tag{9}$$

$$Y_{ik} \in \{0,1\};\ \forall\ \ i \in J, ; k \in K \tag{10}$$

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Identical Machines) – Mean Tardiness Objective

- Equation (1) represents the objective: mean tardiness.
- Constraint set (2) ensures that no job in the sequence can begin processing before time zero.
- Constraint sets (3) and (4) ensure that the processing intervals of a pair of jobs on any machine do not overlap.
- Constraint set (5) allows each job to be assigned to exactly one machine.

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Identical Machines) – Mean Tardiness Objective

- Constraint set (6) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates.

- Constraint sets (7) and (8) ensure that the tardiness and completion times are non-negative.

- Finally, constraint sets (9) and (10) impose a binary restriction on the sequencing variables and assignment variables.

Example

Consider a shop floor with three identical parallel machines. The processing times and due date information for a ten-job order are as follows:

Job #	Pr. Time	Due Date
1	5	8
2	6	10
3	3	12
4	8	15
5	7	8
6	2	5
7	3	3
8	5	7
9	4	6
10	2	10

MILP Solution (Y_{ik} values)

Job i	Machine 1	Machine 2	Machine 3
1	1	0	0
2	0	0	1
3	0	0	1
4	0	1	0
5	1	0	0
6	0	1	0
7	1	0	0
8	0	1	0
9	0	0	1
10	0	1	0

MILP Solution (X_{hi} values)

$h \setminus i$	1	5	7	4	6	8	10	2	3	9
1	-	1	0	-	-	-	-	-	-	-
5	-	-	0	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-	-	-
4	-	-	-	-	0	0	0	-	-	-
6	-	-	-	-	-	1	1	-	-	-
8	-	-	-	-	-	-	1	-	-	-
10	-	-	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	-	1	0
3	-	-	-	-	-	-	-	-	-	0
9	-	-	-	-	-	-	-	-	-	-

MILP Solution

- Optimal sequence:

M1: 7-1-5 | M2: 6-8-10-4 | M3: 9-2-3

Job #	Processing Time	Machine	Completion Time	Due Date	Tardiness
7	3	1	3	3	0
1	5	1	8	8	0
5	7	1	15	8	7
6	2	2	2	5	0
8	5	2	7	7	0
10	2	2	9	10	0
4	8	2	17	15	2
9	4	3	4	6	0
2	6	3	10	10	0
3	3	3	13	12	1
				Total	**10**
				Mean	**1**

MILP Model for Parallel Machine Scheduling (Identical Machines) – Makespan Objective

Objective:

$$Min\ Z = C_{max} \tag{1}$$

subject to

$$C_{max} \geq \sum_{i=1}^{n} Y_{ik} P_i\ ;\ \forall\ \ k \in K \tag{2}$$

$$\sum_{k=1}^{m} Y_{ik} = 1;\ \forall\ \ i \in J \tag{3}$$

$$C_{max} \geq 0; \tag{4}$$

$$Y_{ik} \in \{0,1\};\ \forall\ \ i \in J, ; k \in K \tag{5}$$

MILP Model for Parallel Machine Scheduling (Identical Machines) – Makespan Objective

- Equation (1) represents the objective: makespan.
- Constraint set (2) ensures that the makespan is at least as large as the maximum of the completion times.
- Constraint set (3) allows each job to be assigned to exactly one machine.
- Finally, constraint set (4) imposes a binary restriction on the assignment variables.

Example

Consider a shop floor with three identical parallel machines. The processing times for a ten-job order are as follows:

Job #	Pr. Time
1	5
2	6
3	3
4	8
5	7
6	2
7	3
8	5
9	4
10	2

MILP Solution (Y_{ik} values)

Job i	Machine 1	Machine 2	Machine 3
1	1	0	0
2	0	0	1
3	0	0	1
4	0	1	0
5	1	0	0
6	0	1	0
7	1	0	0
8	0	1	0
9	0	0	1
10	0	0	1

MILP Solution (X_{hi} values)

$h \setminus i$	1	5	7	4	6	8	2	3	9	10
1	-	1	0	-	-	-	-	-	-	-
5	-	-	0	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-	-	-
4	-	-	-	-	0	0	-	-	-	-
6	-	-	-	-	-	1	-	-	-	-
8	-	-	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	1	0	1
3	-	-	-	-	-	-	-	-	0	1
9	-	-	-	-	-	-	-	-	-	1
10	-	-	-	-	-	-	-	-	-	-

MILP Solution

- Optimal sequence:

M1: 7-1-5 | M2: 6-8-4 | M3: 9-2-3-10

Job #	Processing Time	Machine	Completion Time
7	3	1	3
1	5	1	8
5	7	1	15
6	2	2	2
8	5	2	7
4	8	2	15
9	4	3	4
2	6	3	10
3	3	3	13
10	2	2	15
Maximum Completion Time (Makespan)			**15**

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Unrelated Machines)

Notations:

- i, h : indices of job, $i, h \in J = \{1,2, \dots, n\}$
- k : index of machine, $k \in K = \{1,2, \dots, m\}$
- n : number of jobs
- m : number of machines
- P_{ik} : the processing time of job i on machine k
- D_i : due date of job i
- M : A large constant

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Unrelated Machines)

Decision Variables:

- C_i : completion time of job i
- T_i : tardiness of job i
- C_{max} : maximum job completion time (makespan)
- Y_{ik} : assignment variable that takes the value 1, if job i is assigned to machine k, and 0, otherwise
- X_{hi} : sequencing variable that indicates relative job precedence in the processing sequence

$$X_{hi} = \begin{cases} 1, & if\ job\ h\ precedes\ job\ i\ in\ the\ processing\ sequence \\ 0, & otherwise \end{cases}$$

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Unrelated Machines) – Mean Tardiness Objective

Objective:

$$Min\ Z = \frac{1}{n}\sum_{i=1}^{n} T_i \tag{1}$$

subject to

$$C_i \geq \sum_{k=1}^{m} Y_{ik}P_{ik}\ ;\ \forall\ i \in J \tag{2}$$

$$C_i \geq C_h + P_{ik} - M(3 - X_{hi} - Y_{hk} - Y_{ik});\ \forall\ k \in K, ; h, i \in J : h < i \tag{3}$$

$$C_h \geq C_i + P_{hk} - M(2 + X_{hi} - Y_{hk} - Y_{ik});\ \forall\ k \in K, ; h, i \in J : h < i \tag{4}$$

$$\sum_{k=1}^{m} Y_{ik} = 1;\ \forall\ \ i \in J \tag{5}$$

$$T_i \geq C_i - D_i;\ \forall\ \ i \in J \tag{6}$$

$$T_i \geq 0;\ \forall\ \ i \in J \tag{7}$$

$$C_i \geq 0;\ \forall\ \ i \in J \tag{8}$$

$$X_{hi} \in \{0,1\};\ \forall\ \ h, i \in J : h < i \tag{9}$$

$$Y_{ik} \in \{0,1\};\ \forall\ \ i \in J, ; k \in K \tag{10}$$

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Unrelated Machines) – Mean Tardiness Objective

- Equation (1) represents the objective: mean tardiness.
- Constraint set (2) ensures that no job in the sequence can begin processing before time zero.
- Constraint sets (3) and (4) ensure that the processing intervals of a pair of jobs on any machine do not overlap.
- Constraint set (5) allows each job to be assigned to exactly one machine.

Relative Precedence-based MILP Model for Parallel Machine Scheduling (Unrelated Machines) – Mean Tardiness Objective

- Constraint set (6) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates.

- Constraint sets (7) and (8) ensure that the tardiness and completion times are non-negative.

- Finally, constraint sets (9) and (10) impose a binary restriction on the sequencing variables and assignment variables.

Example

Consider a shop floor with three unrelated parallel machines. The processing times and due date information for a ten-job order are as follows:

Job #	Pr. Time on Machine 1	Pr. Time on Machine 2	Pr. Time on Machine 3	Due Date
1	5	8	6	8
2	10	8	6	10
3	5	6	3	12
4	15	8	12	15
5	7	8	8	8
6	4	2	5	5
7	3	3	3	3
8	7	5	6	7
9	5	6	4	6
10	5	2	2	10

MILP Solution (Y_{ik} values)

Job i	Machine 1	Machine 2	Machine 3
1	1	0	0
2	0	0	1
3	0	0	1
4	0	1	0
5	1	0	0
6	0	1	0
7	1	0	0
8	0	1	0
9	0	0	1
10	0	1	0

MILP Solution (X_{hi} values)

$h \setminus i$	1	5	7	4	6	8	10	2	3	9
1	-	1	0	-	-	-	-	-	-	-
5	-	-	0	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-	-	-
4	-	-	-	-	0	0	0	-	-	-
6	-	-	-	-	-	1	1	-	-	-
8	-	-	-	-	-	-	1	-	-	-
10	-	-	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	-	1	0
3	-	-	-	-	-	-	-	-	-	0
9	-	-	-	-	-	-	-	-	-	-

MILP Solution

- Optimal sequence:

M1: 7-1-5 | M2: 6-8-10-4 | M3: 9-2-3

Job #	Processing Time	Machine	Completion Time	Due Date	Tardiness
7	3	1	3	3	0
1	5	1	8	8	0
5	7	1	15	8	7
6	2	2	2	5	0
8	5	2	7	7	0
10	2	2	9	10	0
4	8	2	17	15	2
9	4	3	4	6	0
2	6	3	10	10	0
3	3	3	13	12	1
				Total	**10**
				Mean	**1**

MILP Model for Parallel Machine Scheduling (Unrelated Machines) – Makespan Objective

Objective:

$$Min\ Z = C_{max} \tag{1}$$

subject to

$$C_{max} \geq \sum_{i=1}^{n} Y_{ik} P_{ik}\ ;\ \forall\ k \in K \tag{2}$$

$$\sum_{k=1}^{m} Y_{ik} = 1;\ \forall\ \ i \in J \tag{3}$$

$$C_{max} \geq 0; \tag{4}$$

$$Y_{ik} \in \{0,1\};\ \forall\ \ i \in J, ; k \in K \tag{5}$$

MILP Model for Parallel Machine Scheduling (Unrelated Machines) – Makespan Objective

- Equation (1) represents the objective: makespan.
- Constraint set (2) ensures that the makespan is at least as large as the maximum of the completion times.
- Constraint set (3) allows each job to be assigned to exactly one machine.
- Finally, constraint set (4) imposes a binary restriction on the assignment variables.

Example

Consider a shop floor with three unrelated parallel machines. The processing time information for a ten-job order are as follows:

Job #	Pr. Time on Machine 1	Pr. Time on Machine 2	Pr. Time on Machine 3
1	5	8	6
2	10	8	6
3	5	6	3
4	15	8	12
5	7	8	8
6	4	2	5
7	3	3	3
8	7	5	6
9	5	6	4
10	5	2	2

MILP Solution (Y_{ik} values)

Job i	Machine 1	Machine 2	Machine 3
1	1	0	0
2	0	0	1
3	0	0	1
4	0	1	0
5	1	0	0
6	0	1	0
7	1	0	0
8	0	1	0
9	0	0	1
10	0	0	1

MILP Solution (X_{hi} values)

$h \setminus i$	1	5	7	4	6	8	2	3	9	10
1	-	1	0	-	-	-	-	-	-	-
5	-	-	0	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-	-	-
4	-	-	-	-	0	0	-	-	-	-
6	-	-	-	-	-	1	-	-	-	-
8	-	-	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	1	0	1
3	-	-	-	-	-	-	-	-	0	1
9	-	-	-	-	-	-	-	-	-	1
10	-	-	-	-	-	-	-	-	-	-

MILP Solution

- Optimal sequence:

M1: 7-1-5 | M2: 6-8-4 | M3: 9-2-3-10

Job #	Processing Time	Machine	Completion Time
7	3	1	3
1	5	1	8
5	7	1	15
6	2	2	2
8	5	2	7
4	8	2	15
9	4	3	4
2	6	3	10
3	3	3	13
10	2	2	15
Maximum Completion Time (Makespan)			**15**

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling

Notations:

- i, h : indices of job, $i, h \in J = \{1,2, \dots, n\}$
- k : index of processing stage (machine), $k \in K = \{1,2, \dots, m\}$
- n : number of jobs
- m : number of stages (machines)
- P_{ik} : the processing time of job i in stage k
- D_i : due date of job i
- M : A large constant

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling

Decision Variables:

- C_{ik} : completion time of job i in stage k
- C_{max} : maximum job completion time (makespan)
- T_i : tardiness of job i
- X_{hi} : sequencing variable that indicates relative job precedence in the processing sequence

$$X_{hi} = \begin{cases} 1, & if\ job\ h\ precedes\ job\ i\ in\ the\ processing\ sequence \\ 0, & otherwise \end{cases}$$

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling – Mean Tardiness Objective

Objective:

$$Min\ Z = \frac{1}{n}\sum_{i=1}^{n} T_i \qquad (1)$$

subject to

$$C_{ik} \geq C_{i(k-1)} + P_{ik};\ \forall\ i \in J,\ \ k \in K - \{1\} \qquad (2)$$

$$C_{i1} \geq P_{i1};\ \forall\ i \in J \qquad (3)$$

$$C_{ik} \geq C_{hk} + P_{ik} - M(1 - X_{hi});\ \forall\ k \in K, ; h, i \in J : h < i \qquad (4)$$

$$C_{hk} \geq C_{ik} + P_{hk} - MX_{hi};\ \forall\ k \in K, ; h, i \in J : h < i \qquad (5)$$

$$T_i \geq C_{im} - D_i;\ \forall\ i \in J \qquad (6)$$

$$T_i \geq 0;\ \forall\ i \in J \qquad (7)$$

$$C_{ik} \geq 0;\ \forall\ i \in J,\quad k \in K \qquad (8)$$

$$X_{hi} \in \{0,1\};\ \forall\ h, i \in J : h < i \qquad (9)$$

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling – Mean Tardiness Objective

- Equation (1) represents the objective: mean tardiness.

- Constraint set (2) ensures that, for all operations with a job predecessor, the processing of the kth operation of a job can commence only after the $(k-1)^{th}$ operation of that job has completed processing.

- Constraint set (3) ensures that no job can begin processing before time zero.

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling – Mean Tardiness Objective

- Constraint sets (4) and (5) ensure that the processing intervals of any pair of operations on the same machine do not overlap.

- Constraint set (6) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates, while constraint sets (7) and (8) ensure that the tardiness and completion times are non-negative.

- Finally, constraint set (9) imposes a binary restriction on the sequencing variables.

Example

Processing Times

Jobs	Machine				Due Date
	1	2	3	4	
1	10	15	20	20	70
2	25	5	15	10	80
3	15	10	5	20	75
4	10	15	20	15	65

Solution

X_{hi}	$i = 1$	$i = 2$	$i = 3$	$i = 4$
$h = 1$	-	0	0	0
$h = 2$	-	-	1	0
$h = 3$	-	-	-	0
$h = 4$	-	-	-	-

The optimal sequence is found to be: **4-2-3-1**

Job i	Completion Time, C_i	Due Date, D_i	Tardiness, T_i
4	60	65	0
2	70	80	0
3	90	75	15
1	115	70	45
		Total	60
		Mean	15

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling – Makespan Objective

Objective:

$$Min\ Z = C_{max} \tag{1}$$

subject to

$$C_{ik} \geq C_{i(k-1)} + P_{ik};\ \forall\ \ i \in J,\ \ k \in K - \{1\} \tag{2}$$

$$C_{i1} \geq P_{i1};\ \forall\ i \in J \tag{3}$$

$$C_{ik} \geq C_{hk} + P_{ik} - M(1 - X_{hi});\ \forall\ \ k \in K, ; h, i \in J : h < i \tag{4}$$

$$C_{hk} \geq C_{ik} + P_{hk} - MX_{hi};\ \forall\ \ k \in K, ; h, i \in J : h < i \tag{5}$$

$$C_{max} \geq C_{im};\ \forall\ \ i \in J \tag{6}$$

$$C_{ik} \geq 0;\ \forall\ \ i \in J, \quad k \in K \tag{7}$$

$$X_{hi} \in \{0,1\};\ \forall\ \ h, i \in J : h < i \tag{8}$$

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling – Makespan Objective

- Equation (1) represents the objective: makespan.

- Constraint set (2) ensures that, for all operations with a job predecessor, the processing of the kth operation of a job can commence only after the $(k-1)^{th}$ operation of that job has completed processing.

- Constraint set (3) ensures that no job can begin processing before time zero.

Relative Precedence-based MILP Model for Permutation Flow Shop Scheduling – Makespan Objective

- Constraint sets (4) and (5) ensure that the processing intervals of any pair of operations on the same machine do not overlap.

- Constraint set (6) forces the makespan to be at least as large as the largest job completion time, while constraint set (7) ensures that the completion times are non-negative.

- Finally, constraint set (8) imposes a binary restriction on the sequencing variables.

Example

Processing Times

Jobs	Machine			
	1	2	3	4
1	10	15	20	20
2	25	5	15	10
3	15	10	5	20
4	10	15	20	15

Solution

X_{hi}	$i = 1$	$i = 2$	$i = 3$	$i = 4$
$h = 1$	-	1	0	1
$h = 2$	-	-	0	0
$h = 3$	-	-	-	1
$h = 4$	-	-	-	-

The optimal sequence is found to be: **3-1-4-2**

Job i	Completion Time, C_i
3	50
1	80
4	95
2	**105**

The optimal makespan is found to be: **105 units**

Position-based MILP Model for Permutation Flow Shop Scheduling

Notations:

- i : index of job, $i \in J = \{1,2,\dots,n\}$
- j : index of position, $j \in J = \{1,2,\dots,n\}$
- k : index of machine, $k \in K = \{1,2,\dots,m\}$
- n : number of jobs
- m : number of machines
- P_{ik} : the processing time of job i on machine k
- D_i : due date of job i

Position-based MILP Model for Permutation Flow Shop Scheduling

Decision Variables:

- C_{jk} : completion time of job scheduled in the jth position on machine k
- C_{nm} : makespan, i.e., completion time of job scheduled in the nth (last) position on machine m (last machine)
- T_j : tardiness of job scheduled in the jth position on machine k
- Y_{ij} : sequencing variable that indicates the position of job i in the processing sequence

$$Y_{ij} = \begin{cases} 1, & if\ job\ i\ is\ processed\ in\ the\ j^{th}\ position\ in\ the\ sequence \\ 0, & otherwise \end{cases}$$

Position-based MILP Model for Permutation Flow Shop Scheduling – Mean Tardiness Objective

Objective:

$$Min\ Z = \frac{1}{n}\sum_{j=1}^{n} T_j \tag{1}$$

subject to

$$C_{jk} \geq C_{(j-1)k} + \sum_{i=1}^{n} Y_{ij}P_{ik}\ ;\ \forall\ j \in J - \{1\},\quad k \in K \tag{2}$$

$$C_{jk} \geq C_{j(k-1)} + \sum_{i=1}^{n} Y_{ij}P_{ik}\ ;\ \forall\ j \in J,\quad k \in K - \{1\} \tag{3}$$

$$C_{11} = \sum_{i=1}^{n} Y_{i1}P_{i1} \tag{4}$$

$$\sum_{i=1}^{n} Y_{ij} = 1;\ \ \forall\ j \in J \tag{5}$$

$$\sum_{j=1}^{n} Y_{ij} = 1;\ \ \forall\ i \in J \tag{6}$$

$$T_j \geq C_{jm} - \sum_{i=1}^{n} Y_{ij}D_i\ ;\ \forall\ \ j \in J \tag{7}$$

$$T_j \geq 0;\ \forall\ \ j \in J \tag{8}$$

$$C_{jk} \geq 0;\ \forall\ \ j \in J,\quad k \in K \tag{9}$$

$$Y_{ij} \in \{0,1\};\ \forall\ \ i,j \in J \tag{10}$$

Position-based MILP Model for Permutation Flow Shop Scheduling – Mean Tardiness Objective

- Equation (1) represents the objective: mean tardiness.

- Constraint set (2) ensures that a job can begin processing on a machine only after its predecessor job on the same machine has completed processing.

- Constraint set (3) ensures that a job can begin processing on a machine only after that job has completed processing on the previous machine.

- Constraint set (4) ensures that the job in the first position on the first machine cannot begin processing before time zero.

Position-based MILP Model for Permutation Flow Shop Scheduling – Mean Tardiness Objective

- Constraint sets (5) and (6) ensure that each job is assigned to exactly one position in the sequence and each position is assigned to exactly one job.

- Constraint set (7) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates, while constraint sets (8) and (9) ensure that the tardiness and completion times are non-negative.

- Finally, constraint set (10) imposes a binary restriction on the sequencing variables.

Example

Processing Times

Jobs	Machine				Due Date
	1	2	3	4	
1	10	15	20	20	70
2	25	5	15	10	80
3	15	10	5	20	75
4	10	15	20	15	65

Solution

Y_{ij}	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	0	0	0	1
$i = 2$	0	1	0	0
$i = 3$	0	0	1	0
$i = 4$	1	0	0	0

The optimal sequence is found to be: **4-2-3-1**

Job i	Position j	Completion Time, C_j	Due Date, D_i	Tardiness, T_j
4	1	60	65	0
2	2	70	80	0
3	3	90	75	15
1	4	115	70	45
			Total	60
			Mean	15

Position-based MILP Model for Permutation Flow Shop Scheduling – Makespan Objective

Objective:

$$Min\ Z = C_{nm} \tag{1}$$

subject to

$$C_{jk} \geq C_{(j-1)k} + \sum_{i=1}^{n} Y_{ij} P_{ik}\ ;\ \forall\ j \in J - \{1\},\quad k \in K \tag{2}$$

$$C_{jk} \geq C_{j(k-1)} + \sum_{i=1}^{n} Y_{ij} P_{ik}\ ;\ \forall\ j \in J,\quad k \in K - \{1\} \tag{3}$$

$$C_{11} = \sum_{i=1}^{n} Y_{i1} P_{i1} \tag{4}$$

$$\sum_{i=1}^{n} Y_{ij} = 1;\ \ \forall\ j \in J \tag{5}$$

$$\sum_{j=1}^{n} Y_{ij} = 1;\ \ \forall\ i \in J \tag{6}$$

$$C_{jk} \geq 0;\ \forall\ \ j \in J,\quad k \in K \tag{7}$$

$$Y_{ij} \in \{0,1\};\ \forall\ \ i,j \in J \tag{8}$$

Position-based MILP Model for Permutation Flow Shop Scheduling – Makespan Objective

- Equation (1) represents the objective: makespan.

- Constraint set (2) ensures that a job can begin processing on a machine only after its predecessor job on the same machine has completed processing.

- Constraint set (3) ensures that a job can begin processing on a machine only after that job has completed processing on the previous machine.

Position-based MILP Model for Permutation Flow Shop Scheduling – Makespan Objective

- Constraint set (4) ensures that the job in the first position on the first machine cannot begin processing before time zero.

- Constraint sets (5) and (6) ensure that each job is assigned to exactly one position in the sequence and each position is assigned to exactly one job.

- Constraint set (7) ensures that the tardiness and completion times are non-negative, while constraint set (8) imposes a binary restriction on the sequencing variables.

Example

Processing Times

Jobs	Machine			
	1	2	3	4
1	10	15	20	20
2	25	5	15	10
3	15	10	5	20
4	10	15	20	15

Solution

Y_{ij}	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	0	1	0	0
$i = 2$	0	0	0	1
$i = 3$	1	0	0	0
$i = 4$	0	0	1	0

The optimal sequence is found to be: **3-1-4-2**

Job i	Completion Time, C_i
3	50
1	80
4	95
2	**105**

The optimal makespan is found to be: **105 units**

Alternative Objective Functions in Scheduling

- In addition to the makespan and mean tardiness objectives discussed so far, the following objective functions may also be considered:

1. **Total Flow Time (Mean Flow Time)**
2. **Total Weighted Flow Time (Mean Weighted Flow Time)**
3. **Total Weighted Tardiness (Mean Weighted Tardiness)**

- Also, weighted combinations of two or more objectives can also be considered.

Optional

MILP Model for Job Shop Scheduling (Special case: every job visits every machine exactly once)

Notations:

- i, h : indices of job; $i, h \in J = \{1,2, \dots, n\}$
- j, l : indices of operation of a job; $j, l \in K = \{1,2, \dots, m\}$
- k : index of machine; $k \in K = \{1,2, \dots, m\}$
- n : number of jobs
- m : number of machines (or, number of operations)
- P_{ik} : the processing time of job i on machine k
- y_{ijk} : binary parameter that takes the value 1 when operation j of job i requires machine k, and 0 otherwise
- D_i : due date of job i
- M : A large constant

Optional

MILP Model for Job Shop Scheduling

Decision Variables:

- C_{ik} : completion time of job i on machine k
- T_i : tardiness of job i
- X_{hik} : sequencing variable that indicates relative job precedence in the processing sequence

$$X_{hik} = \begin{cases} 1, & if\ job\ h\ precedes\ job\ i\ on\ machine\ k \\ 0, & otherwise \end{cases}$$

Optional

MILP Model for Job Shop Scheduling

Objective:

$$Min\ Z = \frac{1}{n}\sum_{i=1}^{n} T_i \qquad (1)$$

subject to

$$\sum_{k=1}^{m} C_{ik} y_{ijk} \geq \sum_{k=1}^{m} C_{ik} y_{i(j-1)k} + \sum_{k=1}^{m} P_{ik} y_{ijk}\ ;\ \forall\ i \in J,\ \ j \in K - \{1\} \qquad (2)$$

$$\sum_{k=1}^{m} C_{ik} y_{i1k} \geq \sum_{k=1}^{m} P_{ik} y_{i1k}\ ;\ \forall\ i \in J \qquad (3)$$

$$C_{ik} \geq C_{hk} + P_{ik} - M(1 - X_{hik});\ \forall\ \ k \in K, ; h, i \in J : h < i \qquad (4)$$

$$C_{hk} \geq C_{ik} + P_{hk} - MX_{hik};\ \forall\ \ k \in K, ; h, i \in J : h < i \qquad (5)$$

$$T_i \geq \sum_{k=1}^{m} C_{ik} y_{imk} - D_i;\ \forall\ \ i \in J \qquad (6)$$

$$T_i \geq 0;\ \ \forall\ i \in J \qquad (7)$$

$$C_{ik} \geq 0;\ \ \forall\ i \in J, \quad k \in M \qquad (8)$$

$$X_{hik} \in \{0,1\};\ \ \forall\ k \in K, ;\ h, i \in J : h < i \qquad (9)$$

Optional

MILP Model for Job Shop Scheduling

- Equation (1) represents the objective: mean tardiness.

- Constraint set (2) ensures that, for all operations with a job predecessor, the processing of the jth operation of a job can commence only after the $(j-1)^{th}$ operation of that job has completed processing.

- Constraint set (3) ensures that the first operation of any job cannot begin processing before time zero.

Optional

MILP Model for Job Shop Scheduling

- Constraint sets (4) and (5) ensure that the processing intervals of any pair of operations on the same machine do not overlap.

- Constraint set (6) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates, while constraint sets (7) and (8) ensure that the tardiness and completion times are non-negative.

- Finally, constraint set (9) imposes a binary restriction on the sequencing variables.

Optional

MILP Model for Job Shop Scheduling (General Case)

Notations:

- i, h : indices of job; $i, h \in J = \{1,2, \dots, n\}$
- j, l : indices of operations of job i; $j, l \in N_i = \{1,2, \dots, o_i\}$
- k : index of machine; $k \in K = \{1,2, \dots, m\}$
- n : number of jobs
- o_i : number of operations of job i
- m : number of machines
- P_{ij} : the processing time of operation j of job i
- Q_{ij} : machine assigned to process operation j of job i; $Q_{ij} \in K$
- D_i : due date of job i
- M : A large constant

Optional

MILP Model for Job Shop Scheduling

Decision Variables:

- C_{ij} : completion time of operation j of job i
- T_i : tardiness of job i
- X_{hlij} : sequencing variable that indicates relative operation precedence in the processing sequence

$$X_{hlij} = \begin{cases} 1, & if\ operation\ l\ of\ job\ h\ precedes\ operation\ j\ of\ job\ i \\ 0, & otherwise \end{cases}$$

Optional

MILP Model for Job Shop Scheduling

Objective:

$$Min\ Z = \frac{1}{n}\sum_{i=1}^{n} T_i \tag{1}$$

subject to

$$C_{ij} \geq C_{i(j-1)} + P_{ij};\ \forall\ i \in J,\ j \in N_i - \{1\} \tag{2}$$

$$C_{i1} \geq P_{i1};\ \forall\ i \in J \tag{3}$$

$$C_{ij} \geq C_{hl} + P_{ij} - M\left(1 - X_{hlij}\right);\ \forall\ l \in N_h,\ j \in N_i,\ h, i \in J : h < i,\ Q_{hl} = Q_{ij} \tag{4}$$

$$C_{hl} \geq C_{ij} + P_{hl} - MX_{hlij};\ \forall\ l \in N_h,\ j \in N_i,\ h, i \in J : h < i,\ Q_{hl} = Q_{ij} \tag{5}$$

$$T_i \geq C_{io_i} - D_i;\ \forall\ i \in J \tag{6}$$

$$T_i \geq 0;\ \forall\ i \in J \tag{7}$$

$$C_{ij} \geq 0;\ \forall\ i \in J,\ j \in N_i \tag{8}$$

$$X_{hlij} \in \{0,1\};\ \forall\ l \in N_h,\ j \in N_i,\ h, i \in J : h < i,\ Q_{hl} = Q_{ij} \tag{9}$$

Optional

MILP Model for Job Shop Scheduling

- Equation (1) represents the objective: mean tardiness.

- Constraint set (2) ensures that, for all operations with a job predecessor, the processing of the jth operation of a job can commence only after the $(j-1)^{th}$ operation of that job has completed processing.

- Constraint set (3) ensures that the first operation of any job cannot begin processing before time zero.

Optional

MILP Model for Job Shop Scheduling

- Constraint sets (4) and (5) ensure that the processing intervals of any pair of operations on the same machine do not overlap.

- Constraint set (6) links the decision variables corresponding to the tardiness of jobs to their completion times and due dates, while constraint sets (7) and (8) ensure that the tardiness and completion times are non-negative.

- Finally, constraint set (9) imposes a binary restriction on the sequencing variables.

Thank You !!!